Tudo o que aprendemos criando jogos no Roblox

Dog Fighter Simulator · atualizado em outubro de 2026

1. Como trabalhar

- **Planejar antes de programar.** Mudanças grandes (economia, jogador virar o cachorro) começam com um plano aprovado pelo dono e são feitas **em partes**, testando cada uma.
- **Uma pergunta por vez, com opções.** Decisões de design (quantas vagas, onde fica o estande) são do dono; o resto segue um padrão sensato.
- **Sempre rodar a checagem e ler a saída INTEIRA.** Um erro passou despercebido porque só olhei o "issues: 0" do final, e a tela de vários ovos quebrou.
- **Revisão de segurança** com o agente revisor depois de qualquer sistema com itens valiosos (trade, compras, economia). Ele achou problemas reais nas duas vezes.
- **Entregar pronto para usar:** um único `.bat` que publica tudo, arquivos mandados direto (o dono não consegue abrir pastas nem achar IDs).
- **O ambiente de trabalho é temporário:** guardar o `roblox-dev.md` (agente) e o `PROJETO_COMPLETO_DogFighter.zip` (jogo) depois de cada sessão.

2. Organização do código

- **Rojo:** o código fica em arquivos e vira o `.rbxl`. Pastas: `server` (regras, dinheiro, save), `client` (telas, sons, controles), `shared` (configurações e IDs).
- **Todos os números do jogo num** `Config.lua` (preços, XP, tempos). Balancear = trocar números.
- **O servidor decide tudo o que vale dinheiro ou Robux.** O cliente só pede, e com limite de pedidos (throttle).
- Entre serviços do servidor, usar um **Signal em Lua**, nunca `BindableEvent` (ele copia tabelas e quebrou os cachorros mortos que não sumiam).
- A função `Net.Invoke` só devolve 2 valores → para devolver mais coisa, usar **uma tabela**.
- Tudo que pode falhar (DataStore, compras, PolicyService, rankings) vai dentro de `pcall`.
- Objetos temporários (corpos, efeitos) **se apagam sozinhos**, e apagar duas vezes não pode dar erro.

3. Save (dados do jogador)

- Campo novo no save → criar com **valor padrão na sanitização**, senão jogadores antigos quebram.
- **Migração:** a sanitização copia os campos do modelo antes de tudo. Um campo que precisa ser calculado a partir de dados antigos (ex.: estande a partir das tigelas) **não pode estar no modelo**, senão a migração nunca roda.
- Itens que deixaram de existir (ovos do ninho) são **convertidos** (foram para o inventário), nunca apagados.
- Timers **não acumulam offline** (no máximo 1 recompensa ao voltar).

4. O jogador é o cachorro

- O modelo do cachorro (com `Humanoid` e uma peça `HumanoidRootPart`) vira o `player.Character`. O avatar só aparece antes de escolher o primeiro cachorro (`CharacterAutoLoads = false`).

- O **jogador controla a física** do cachorro. Quando o servidor precisa controlar (poder especial, torre do boss), ele pega a física de volta e o cliente **trava os controles**; no fim, devolve.
- Códigos que reagem a "personagem novo" (mandar para casa, tirar da torre) precisam **ignorar quando o personagem novo é o cachorro**.
- Trocar o personagem pelo servidor **não move a câmera sozinho**: um guard no cliente sempre põe a câmera no cachorro atual.
- **Correr com Shift:** desligar o Shift Lock do Roblox. A velocidade é a do cachorro × 1,6. No celular, botão RUN.
- **Cachorros com asas** ficam no chão; **espaço 2x** (ou pular 2x no celular) liga/desliga o voo.

5. Combate controlado pelo jogador

- Habilidades por um remote só (`UseAbility`): **1 = Mordida**, **2 = Poder especial**, **3 = poder do tipo** (em breve). O **servidor** escolhe o alvo (o mais perto na frente), o dano e a recarga.
- Reaproveitar a IA antiga: morder um cachorro **começa a luta** e o NPC revida sozinho. Poderes que movem o cachorro rodam com o servidor no controle por 1–2 s.
- PvP só quando **os dois** ligaram o PvP. Sair da Arena andando encerra a luta.
- Barra de habilidades: PC com teclas e clique; celular com 3 botões redondos; recarga visível.

6. Economia (estilo simulador)

- Ciclo: **ganhar dinheiro** → **comprar ovo** → **cachorro melhor** → **ganhar mais.**
- **Fontes de dinheiro:** time de cachorros que rende $/min (2 vagas → até 6 compradas), Arena e Boss, vender repetidos (e, nas próximas partes, ossos/baús e rebirth).
- **Gastos:** ovos à venda (Paw $100 → Celestial $300k), estande de treino (5 materiais × 3 níveis), vagas no time, casinha (+10% de renda por nível); depois petiscos, fundir, áreas novas e rebirth.
- **Rebirth** (planejado): zera dinheiro e níveis dos cachorros, mantém os cachorros, dá +50% de dinheiro para sempre.
- **Comprar = cobrar e guardar o item antes de abrir.** Se não der para abrir (canil cheio), avisar que ficou guardado.
- Botão de clique para bônus (TRAIN) **não conta na missão de batidas** (senão autoclicker termina a missão), e o "treinando" desliga quando o cachorro sai de perto.
- **Progressão:** o XP necessário cresce mais depois do nível 15; os jogadores avançavam rápido demais.
- **Vender raridades altas:** só os repetidos, sempre guardando o melhor de cada espécie.

7. Trade entre jogadores

- A troca vive **no servidor**. Qualquer mudança na oferta **tira o READY dos dois** e cancela a contagem (anti-golpe).
- Na hora de trocar, **confere tudo de novo** e mexe nos dois perfis no mesmo passo.
- Não troca o cachorro equipado nem os favoritos; o canil precisa caber; o cachorro chega com o nome padrão.
- Limite de pedidos por jogador (ninguém spamma pedidos). Risco raro aceito: se o servidor cair no segundo da troca.

8. Interface e celular

- **A maioria joga no celular:** escalar pela tela, banners e avisos menores e curtos (o level up tampava a tela).

- Telas importantes (ovo, primeira vez) ficam **acima do tutorial**, e o tutorial se esconde enquanto elas estão abertas.
- **Ações em massa** (170 ovos): grade de 10 + CONTINUE + **SKIP ALL** fixo embaixo, que para a animação na hora.
- **Tutorial:** pouco texto, seta no botão, caminho no chão, marcador no objeto, skip e prêmio no fim.
- Menu de **Settings** (gráficos, música, efeitos) salvo na conta. Jogo em **inglês**.

9. Áudio

- Limite de uploads de áudio por mês → **todos os efeitos num único áudio**, tocando pedaços (`PlaybackRegion`).
- Sons de luta em 3D, sons de botão em 2D; música diferente para lobby, arena e boss, mais jingles.

10. Gamepasses e Robux

- **Primeiro publique o jogo, depois crie os passes nele.** Os passes pertencem ao jogo: publicar como jogo novo perde os passes.
- Gamepass = compra uma vez; Developer Product = compra várias vezes. Ícones 512×512.
- Quando um sistema muda (ovos não nascem mais), **trocar a função do passe** (Fast Eggs → Fast Training) e **renomear no site**.
- **Itens aleatórios pagos:** checar `ArePaidRandomItemsRestricted` e bloquear para quem tem restrição; responder no questionário.
- DevEx exige no mínimo 30.000 Robux ganhos.

11. Publicação

- Studio preso em "Upload in progress" → reiniciar, desligar Team Create, ou publicar pelo `.bat` (Open Cloud).
- Chave do Open Cloud: precisa do jogo **e** das operações de escrita (erro 403 = faltou operação). IP `0.0.0.0/0`. **A chave é uma senha:** não compartilhar o `.bat`.
- A API não cria jogo novo, só atualiza.
- Max Players = capacidade do mapa (8 casinhas). Ligar "Enable Studio Access to API Services" para o save funcionar no Studio.
- **Testar no Studio antes de publicar** mudanças grandes (Play, emulador de celular, 2 jogadores).

12. Público e retenção

- Responder o questionário de maturidade com sinceridade; a taxa de publicação reembolsável influencia o alcance para públicos mais novos. Conferir a página **Audience**.
- O Roblox recomenda jogos com **retenção** e **tempo de jogo**: coleção, rankings semanais, missões, bosses, sempre um próximo objetivo.
- Qualidade de vida conta: venda automática, venda rápida, imagem de cada item, skip de animações.